

Metodologie di Programmazione (A-L)

Il semestre - a.a. 2025 – 2026

Istruzioni di Controllo

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Istruzioni di Controllo

Istruzioni di controllo

```
public static void main(String[] args)
{
    int x = 5;
    int y = 7;

    x = y+1;
    y = x+1;
}

↔

public static void main(String[] args)
{
    int x = 5;
    int y = 7;

    y = x+1;
    x = y+1;
}
```

- Finora abbiamo **controllato** il **flusso** di esecuzione del programma solo mediante la **sequenza**
- La **sequenza** è la **struttura di controllo** fondamentale che stabilisce l'ordine di esecuzione delle istruzioni

Prendere decisioni

- Come prendere decisioni in Java?
- Mediante le **istruzioni di controllo condizionali**
 - Alcune istruzioni possono essere o non essere eseguite sulla base di **certe condizioni**
- Mediante le **istruzioni di controllo iterative**
 - Permettono di specificare che un blocco di istruzioni deve **essere eseguito ripetutamente** sulla base di **certe condizioni**

L'istruzione if

- Per realizzare una **decisione** si usa l'istruzione **if**
- La **sintassi** è:

```
if (<espressione booleana>) <singola istruzione>
```

oppure:

```
if (<espressione booleana>)  
{  
    <istruzioni>  
}
```

Eseguite se l'espressione booleana è **vera**

- Esempi:

```
if (x < 0) x = -x;
```

```
if (x < y)  
{  
    int t = x;  
    x = y;  
    y = t;  
}
```

Codificare l'alternativa: l'istruzione else

- Per specificare l'alternativa da eseguire nel caso in cui il valore dell'espressione booleana sia falso
- Sintassi:

```
if (<espressione booleana>)  
{  
    <istruzioni caso true>  
}  
else  
{  
    <istruzioni caso false>  
}
```

Eseguite se l'espressione booleana è vera

Eseguite se l'espressione booleana è falsa

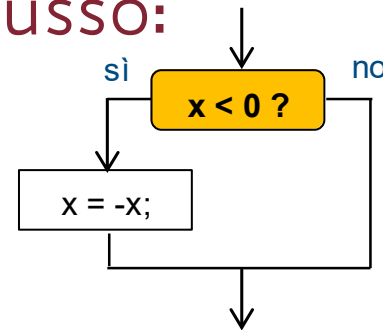
- Esempio

```
:   if (x > y)   max = x;  
    else       max = y;
```

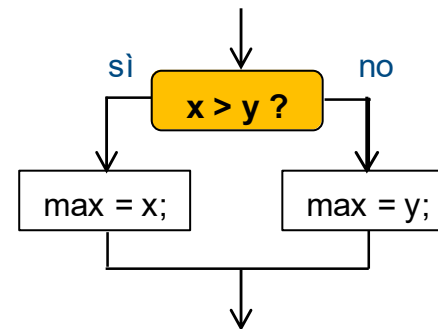
Enunciati if ed else mediante diagrammi di flusso

- Per capire il flusso del programma controllato mediante le istruzioni **if** ed **else**, possiamo utilizzare i **diagrammi di flusso**:

```
if (x < 0) x = -x;
```



```
if (x > y) max = x;  
else      max = y;
```



Esempi

valore assoluto

```
if (x < 0) x = -x;
```

ordina x e y

```
if (x < y)
{
    int t = x;
    x = y;
    y = t;
}
```

massimo di x e y

```
if (x > y) max = x;
else      max = y;
```

controllo di divisione
per zero

```
if (den == 0)    System.out.println("Division by zero");
else            System.out.println("Quoziente = " + num/den);
```

calcolo delle radici
di un'equazione di
secondo grado
 $x^2+bx+c = 0$

```
double discriminant = b*b - 4.0*c;
if (discriminant < 0.0) System.out.println("Non ci sono radici reali");
else
{
    System.out.println((-b + Math.sqrt(discriminant))/2.0);
    System.out.println((-b - Math.sqrt(discriminant))/2.0);
}
```

L'else "sospeso"

```
if (x > 0)
    if (y > 0)
        System.out.println("x e y sono > 0");
else
    System.out.println("x <= 0");
```

Errore: non si riferisce a if (x > 0)
ma a if (y > 0)!!!

```
if (x > 0)
    if (y > 0)
        if (z < 5)
            if (w >= 3)
                System.out.println("x e y sono > 0, z < 5 e w >= 3");
            else
                System.out.println("x e y sono > 0, z < 5 e w < 3");
```

Corretto!

E' importante essere consapevoli che l'**else** si riferisce SEMPRE all'istruzione **if** immediatamente precedente

Forzare il riferimento dell'else all'istruzione if richiesta

- Utilizzando le **parentesi graffe** per specificare il corpo dell'if precedente

```
if (x > 0)
{
    if (y > 0)
        System.out.println("x e y sono > 0");
}
else
    System.out.println("x <= 0");
```

Tirare un moneta (non truccata) in due righe!

```
public class TiraMoneta
{
    public static void main(String[] args)
    {
        if (Math.random() < 0.5)    System.out.println("Testa");
        else                        System.out.println("Croce");
    }
}
```

• Esempio di esecuzione:

- java TiraMoneta
Testa
- java TiraMoneta
Croce
- java TiraMoneta
Croce
- ...

Operatore «Elvis» di selezione (o operatore condizionale)

- In Java (come in C) esiste un operatore di selezione (operatore condizionale) nella forma di espressione condizione ? valoreCasoVero : valoreCasoFalso
- Esempi:

```
int abs = x < 0 ? -x : x;
```

```
int max = x > y ? x : y;
```

```
String testaCroce = Math.random() < 0.5 ? "Testa" : "Croce";
```

Esercizio

- Progettare un metodo che **estrae il carattere centrale** da una stringa fornita in input
- Se la stringa ha un **numero pari di caratteri**, estrarre i **due caratteri centrali** (es. da "magico" deve estrarre "gi")
- **Fase 1:** progettare l'interfaccia pubblica

```
public String estraiCarattereCentrale(String s)
{
}
```

- **Fase 2:** individuare la **condizione per la diramazione**:
 - La lunghezza della stringa è **dispari**?
 - `s.length() % 2 == 1`?

Esercizio

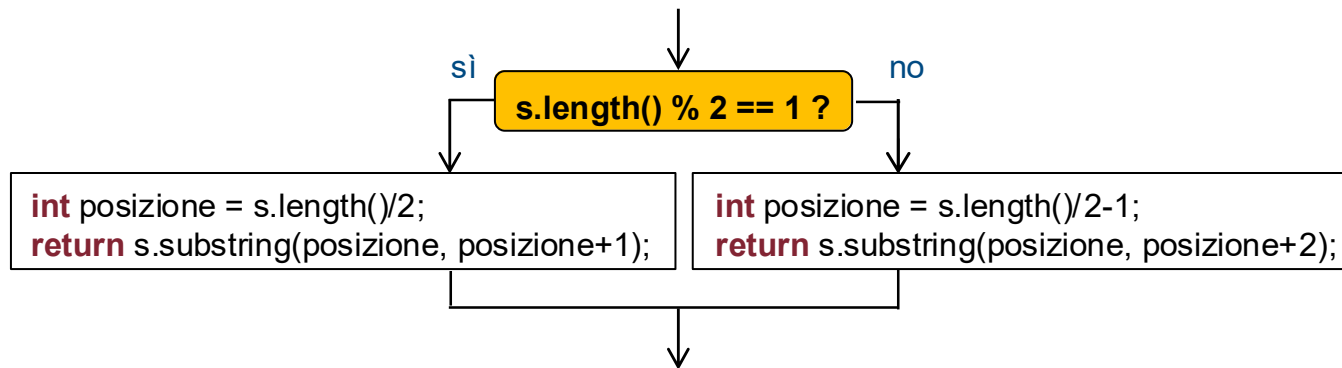
- Fase 3: progettare il codice (o lo **pseudocodice**) per le istruzioni da eseguire quando la condizione è soddisfatta

```
int posizione = s.length()/2;  
return s.substring(posizione, posizione+1);
```

- Fase 4: progettare il codice (o lo **pseudocodice**) per le istruzioni da eseguire quando la condizione **NON** è soddisfatta

```
int posizione = s.length()/2-1;  
return s.substring(posizione, posizione+2);
```

- Progetta il **flusso di controllo**:



Esercizio

- Una prima versione:

```
public String estraiCarattereCentrale(String s)
{
    if (s.length() % 2 == 1)
    {
        int posizione = s.length()/2;
        return s.substring(posizione, posizione+1);
    }
    else
    {
        int posizione = s.length()/2-1;
        return s.substring(posizione, posizione+2);
    }
}
```

Esercizio

- Una prima versione:
- Fase 4: elimina le ripetizioni

```
public String estraiCarattereCentrale(String s)
{
    if (s.length() % 2 == 1)
    {
        int posizione = s.length()/2;
        return s.substring(posizione, posizione+1);
    }
    else
    {
        int posizione = s.length()/2-1;
        return s.substring(posizione, posizione+2);
    }
}
```

```
public String estraiCarattereCentrale(String s)
{
    int posizione;
    int lunghezza;

    if (s.length() % 2 == 1)
    {
        posizione = s.length()/2;
        lunghezza = 1;
    }
    else
    {
        posizione = s.length()/2-1;
        lunghezza = 2;
    }

    return s.substring(posizione, posizione+lunghezza);
}
```

Alternative multiple

- In molte situazioni è necessario prendere **più di una decisione** di tipo if/else
- E' richiesta una **sequenza di confronti**

Esempio: descrizione del terremoto

- Progettare una classe che restituisca una descrizione del terremoto dato il valore di magnitudo della scossa su scala Richter

```
public class Terremoto
{
    /**
     * Valore sulla scala Richter
     */
    private double magnitudo;

    public Terremoto(double magnitudo)
    {
        this.magnitudo = magnitudo;
    }

    public String toString()
    {
        if (magnitudo >= 8.0) return "La maggior parte delle strutture saranno distrutte";
        else if (magnitudo >= 7.0) return "Molti edifici saranno distrutti";
        else if (magnitudo >= 6.0) return "Molti edifici saranno danneggiati, alcuni distrutti";
        else if (magnitudo >= 4.5) return "Danni solo a edifici con struttura debole";
        else if (magnitudo >= 3.5) return "Terremoto percettibile, ma nessuna distruzione";
        else if (magnitudo >= 0) return "Non percettibile";
        else return "Valore negativo non ammesso";
    }
}
```

Alternative multiple: l'istruzione switch

- Per confrontare il valore di un'espressione intera o convertibile a intero (o, da Java 7 in poi, un valore stringa), si può usare l'istruzione **switch**:

```
switch(<espressione intera>
{
    case <valore1>: <istruzioni>; break;
    case <valore2>: <istruzioni>; break;
    ...
    case <valoren>: <istruzioni>; break;
}
```

Esercizio: Saluto casuale

- Progettare un metodo che emetta sullo **standard output** un saluto scelto **casualmente** tra "ciao", "hello", "bella", "salve" e "buongiorno" (rendere quest'ultimo doppiamente più probabile)

```
import java.util.Random;

public class SalutoCasuale
{
    public void sayHello()
    {
        String hello = null;

        // new Random().nextInt(6) equivalente a (int)(Math.random()*6)
        switch(new Random().nextInt(6))
        {
            case 0:    hello = "ciao"; break;           // informale
            case 1:    hello = "hello"; break;         // inglese
            case 2:    hello = "bella"; break;         // romanesco
            case 3:    hello = "salve"; break;         // salute a te
            default:   hello = "buongiorno"; break;    // formale
        }

        System.out.println(hello);
    }

    public static void main(String[] args)
    {
        new SalutoCasuale().sayHello();
    }
}
```



caso di default

Switch compatti (java >=13)

- E' possibile utilizzare anche una notazione contratta con l'operatore `->` che non richiede l'utilizzo del `break` per uscire:

```
switch(k % 7)
{
    case 0:
        iniziaLaSettimana();
        break;
    case 1:
        faiLaSpesa();
        break;
    /* ... */
    case 5:
        relax();
        break;
    case 6:
        gitaDellaDomenica();
        break;
}
```

java >=13

```
switch(k % 7)
{
    case 0 -> iniziaLaSettimana();
    case 1 -> faiLaSpesa();
    /* ... */
    case 5 -> relax();
    case 6 -> gitaDellaDomenica();
}
```

Espressioni switch (java >=13)

- E' possibile utilizzare lo switch con la notazione -> facendo restituire un'espressione:

java >=13

```
String s;  
switch(c)  
{  
    case 'k': s = "kappa"; break;  
    case 'h': s = "acca"; break;  
    case 'l': s = "elle"; break;  
    case 'c': s = "ci"; break;  
    case 'a': case 'e': case 'i':  
    case 'u': case 'o':  
        s = "vocale "+c; break;  
    default: s = "non so leggerlo";  
}
```

```
String s = switch(c) {  
    case 'k' -> "kappa";  
    case 'h' -> "acca";  
    case 'l' -> "elle";  
    case 'c' -> "ci";  
    case 'a', 'e', 'i',  
        'u', 'o' -> "vocale "+c;  
    default -> "non so leggerlo";  
};
```

Espressioni switch (java >=13)

- La parola chiave `yield` permette di utilizzare lo switch classico come espressione e restituire il valore di interesse (interrompendo l'esecuzione dello switch):

java >=13

```
String s;  
switch(c)  
{  
    case 'k': s = "kappa"; break;  
    case 'h': s = "acca"; break;  
    case 'l': s = "elle"; break;  
    case 'c': s = "ci"; break;  
    case 'a': case 'e': case 'i':  
    case 'u': case 'o':  
        s = "vocale "+c; break;  
    default: s = "non so leggerlo";  
}
```

```
String s = switch(c) {  
    case 'k': yield "kappa";  
    case 'h': yield "acca";  
    case 'l': yield "elle";  
    case 'c': yield "ci";  
    case 'a': case 'e': case 'i':  
    case 'u': case 'o':  
        System.out.println("vocale!");  
        yield "vocale "+c;  
    default: yield "non so leggerlo";  
};
```

La parola chiave var (java >=10)

- A partire da Java 10, è possibile **evitare di dichiarare il tipo** di una variabile locale

```
var k = 10;           // equivale a int k = 10;  
var s = "ciao";       // equivale a String s = "ciao";  
var p = new Pippo(); // equivale a Pippo p = new Pippo();
```

- Il tipo della variabile è sempre prefissato e **non può cambiare**
- E' possibile utilizzare **var** solo per le variabili locali
 - Non per i campi, né per i parametri di metodi
- **Consiglio:** la gestione statica dei tipi di Java è un punto di forza -> **utilizzate var solo per le variabili di comodo**

Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli.

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischi, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.